

Modelos Avanzados de Computación Examen I

FACULTAD
DE
CIENCIAS
UNIVERSIDAD DE GRANADA



Los Del DGIIM, losdeldgiim.github.io

Doble Grado en Ingeniería Informática y Matemáticas
Universidad de Granada



Esta obra está bajo una Licencia Creative Commons Atribución-NoComercial-SinDerivadas 4.0 Internacional (CC BY-NC-ND 4.0).

Eres libre de compartir y redistribuir el contenido de esta obra en cualquier medio o formato, siempre y cuando des el crédito adecuado a los autores originales y no persigas fines comerciales.

Modelos Avanzados de Computación Examen I

Los Del DGIIM, losdeldgiim.github.io

José Manuel Sánchez Varbas

Granada, 2026

Asignatura Modelos Avanzados de Computación.

Curso Académico 2024-25.

Grado Grado en Ingeniería Informática.

Descripción Ordinaria.

Fecha 11 de Junio de 2025.

Ejercicio 1 (2.4 puntos). Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles, justificando las respuestas (se supone que las MTs tienen a $\{0, 1\}$ como alfabeto de entrada):

1. Dada una MT M de dos cintas y una palabra u de longitud menor o igual a 100, ¿es cierto que nunca usa más de 100 casillas en la primera cinta cuando lee la palabra u ?
2. Dada una MT M , ¿es cierto que acepta todas las palabras de longitud menor o igual a 10?
3. Dadas dos MT, M_1 y M_2 , ¿es cierto que toda palabra de longitud menor o igual a 100 que es aceptada por M_1 lo es también por M_2 con un número de pasos menor o igual al empleado por M_1 ?
4. Dadas dos MT, M_1 y M_2 , y una palabra u , donde M_2 está garantizado que termina para todas las entradas, ¿es cierto que M_1 da más pasos que M_2 cuando ambas leen la palabra u ?

Ejercicio 2 (1.3 puntos). Describe de forma algorítmica una MT que acepte las palabras que son de la forma uu , donde $u \in \{0, 1\}^*$ (es decir que sea una palabra repetida dos veces) en espacio logarítmico.

Ejercicio 3 (2.4 puntos). Razona si son verdaderas o falsas las siguientes afirmaciones:

1. Una MT con una cinta ilimitada en ambas direcciones se puede simular siempre con una MT con una cinta ilimitada solo por la derecha y usando el mismo número de pasos.
2. Para que una MT tenga una complejidad en espacio de $O(n^2)$ tiene que tener más de una cinta para no escribir en la cinta de entrada.
3. Nuestro conocimiento actual es compatible con el hecho de que $\mathbf{P} \neq \mathbf{NP}$ y $\mathbf{NP} = \mathbf{CoNP}$.
4. Está demostrado que la tesis de Church-Turing es cierta.

Ejercicio 4 (1.3 puntos). Enuncia los problemas de la suma. ¿Qué sabes sobre su complejidad? Razona de forma breve la respuesta.

Ejercicio 5 (1.3 puntos). Define la clase **EXP**, ¿qué relación existe con las clases **PESPACIO** y **P**?

Ejercicio 6 (1.3 puntos). Elige entre una de las siguientes opciones:

- Describe una reducción Turing de $\text{FSAT}(C)$ a $\text{SAT}(C)$.
- Sean los problemas de la existencia de un circuito hamiltoniano, $\text{HAMILTONIANO}(G)$, y el problema de buscar un circuito hamiltoniano cuando este exista, $\text{FHAMILTONIANO}(G)$. Describe una reducción Turing de $\text{FHAMILTONIANO}(G)$ a $\text{HAMILTONIANO}(G)$.

Solución.

Ejercicio 1 (2.4 puntos). Determinar cuáles de los siguientes problemas son decidibles, semidecidibles o no semidecidibles, justificando las respuestas (se supone que las MTs tienen a $\{0, 1\}$ como alfabeto de entrada):

1. Dada una MT M de dos cintas y una palabra u de longitud menor o igual a 100, ¿es cierto que nunca usa más de 100 casillas en la primera cinta cuando lee la palabra u ?

Definimos

$$P_1 = \{\langle M, u \rangle : |u| \leq 100 \text{ y n}^\circ \text{ casillas distintas 1}^\circ \text{ cinta visitadas por } M \text{ con entrada } u \leq 100\}$$

Veamos que no es semidecidible. Reducimos desde

$$C\text{-PARADA} = \{\langle M, w \rangle : M \text{ no para con entrada } w\},$$

que sabemos que no es semidecidible. Dada una instancia $\langle M, w \rangle$, construimos una MT de dos cintas N y fijamos $u_0 = \varepsilon$.

La máquina N , con entrada u_0 , simula M con entrada w usando solo la segunda cinta. Si la simulación de $M(w)$ para, entonces N mueve el cabezal de la primera cinta hasta visitar 101 casillas distintas.

Si M no para con entrada w , entonces N nunca visita más de 100 casillas distintas en la primera cinta, luego

$$\langle N, u_0 \rangle \in P_1.$$

Si M para con entrada w , entonces N acaba visitando 101 casillas distintas de la primera cinta, luego

$$\langle N, u_0 \rangle \notin P_1.$$

Por tanto,

$$\langle M, w \rangle \in C\text{-PARADA} \iff \langle N, u_0 \rangle \in P_1.$$

Luego $C\text{-PARADA} \propto P_1$. Como $C\text{-PARADA}$ no es semidecidible, P_1 tampoco es semidecidible. En particular, no es decidable.

2. Dada una MT M , ¿es cierto que acepta todas las palabras de longitud menor o igual a 10?

Definimos

$$P_2 = \{\langle M \rangle : \forall w \in \{0, 1\}^*, |w| \leq 10 \Rightarrow w \in L(M)\}.$$

Este problema no es decidible por el Teorema de Rice, ya que la propiedad “aceptar todas las palabras de longitud menor o igual que 10” depende solo del lenguaje $L(M)$, no de la máquina concreta. Además, es no trivial: la cumple una MT que acepta $\{0, 1\}^*$, pero no la cumple una MT que no acepta ninguna palabra.

Veamos que sí es semidecidible. Sea

$$A = \{w \in \{0, 1\}^* : |w| \leq 10\}.$$

Como A es finito, podemos escribir

$$A = \{w_1, \dots, w_k\}.$$

para cierto $k \in \mathbb{N}$. Construimos una MTND N que funciona como sigue:

Algoritmo 1 MTND N

Entrada: Una codificación $\langle M \rangle$ de una MT M .

Sea $A = \{w \in \{0, 1\}^* : |w| \leq 10\} = \{w_1, \dots, w_k\}$.

Selecciona de forma no determinista un número natural t .

Para $i = 1, \dots, k$ **hacer**

 Simulamos M con entrada w_i durante t pasos.

Si M no acepta w_i en esos t pasos **entonces**

N rechaza.

Fin Si

Fin Para

N acepta.

Si M acepta todas las palabras de longitud menor o igual que 10, entonces para cada w_i existe un número de pasos t_i en el que M acepta w_i . Por tanto, existe una rama de cómputo de N que elige precisamente esos tiempos $t_1, \dots, t_k$, y esa rama acepta.

Si M no acepta alguna palabra w_i de longitud menor o igual que 10, entonces,elijamos los tiempos que elijamos, M no aceptará esa palabra en esos pasos, y ninguna rama podrá aceptar.

Por tanto, P_2 es semidecidible pero no decidible.

3. Dadas dos MT, M_1 y M_2 , ¿es cierto que toda palabra de longitud menor o igual a 100 que es aceptada por M_1 lo es también por M_2 con un número de pasos menor o igual al empleado por M_1 ?

Definimos

$$P_3 = \{\langle M_1, M_2 \rangle : \forall z, |z| \leq 100, \text{ si } M_1 \text{ acepta } z \text{ en } t_1 \text{ pasos, entonces } M_2 \text{ acepta } z \text{ en } t_2 \leq t_1 \text{ pasos}\}.$$

Veamos que P_3 no es semidecidible. Reducimos desde

$$C\text{-UNIVERSAL}(M, w) = \{\langle M, w \rangle : M \text{ no acepta } w\},$$

que sabemos que no es semidecidible. Dada una instancia $\langle M, w \rangle$, construimos dos MT N_1 y N_2 :

Algoritmo 2 MT N_1

Entrada: Una palabra $x \in \{0, 1\}^*$.
Si $x \neq \varepsilon$ **entonces**
 Rechazar.
Fin Si
Simular M con entrada w .
Si M acepta w **entonces**
 Aceptar.
Fin Si

Algoritmo 3 MT N_2

Entrada: Una palabra $x \in \{0, 1\}^*$.
Rechazar.

Si M no acepta w , entonces N_1 no acepta ninguna palabra. Por tanto, la condición de P_3 se cumple trivialmente, ya que no hay palabras aceptadas por N_1 .

Si M acepta w , entonces N_1 acepta ε en cierto número t_1 de pasos, pero N_2 no acepta ε en ningún número de pasos. Por tanto, la condición de P_3 falla.

Así,

$$\langle M, w \rangle \in C\text{-UNIVERSAL} \iff \langle N_1, N_2 \rangle \in P_3.$$

Luego $C\text{-UNIVERSAL} \propto P_3$. Como $C\text{-UNIVERSAL}$ no es semidecidible, P_3 no es semidecidible. En particular, P_3 no es decidable.

4. Dadas dos MT, M_1 y M_2 , y una palabra u , donde M_2 está garantizado que termina para todas las entradas, ¿es cierto que M_1 da más pasos que M_2 cuando ambas leen la palabra u ?

Definimos

$$P_4 = \{\langle M_1, M_2, u \rangle : M_1 \text{ da más pasos que } M_2 \text{ con entrada } u\}.$$

Este problema es decidible. Como M_2 está garantizado que termina para todas las entradas, en particular termina con entrada u . Por tanto, podemos simular $M_2(u)$ hasta que pare y calcular su número de pasos, digamos t_2 .

Después simulamos $M_1(u)$ durante exactamente t_2 pasos.

Algoritmo 4 M_{P_4}

Entrada: Una codificación $\langle M_1, M_2, u \rangle$.

Simular M_2 con entrada u hasta que termine.

Sea t_2 el número de pasos dados por M_2 .

Simular M_1 con entrada u durante exactamente t_2 pasos.

Si M_1 no ha terminado en ningún momento de esos t_2 pasos **entonces**

Aceptar.

Si no

Rechazar.

Fin Si

El algoritmo siempre termina, porque $M_2(u)$ termina por hipótesis y luego solo se simula $M_1(u)$ durante un número finito de pasos. Además, si M_1 no ha terminado en los primeros t_2 pasos, entonces da más pasos que M_2 ; mientras que si ha terminado en algún instante $t_1 \leq t_2$, no da más pasos que M_2 .

Por tanto, P_4 es decidible.

Ejercicio 2 (1.3 puntos). Describe de forma algorítmica una MT que acepte las palabras que son de la forma uu , donde $u \in \{0, 1\}^*$ (es decir que sea una palabra repetida dos veces) en espacio logarítmico.

Sea x la palabra de entrada y sea $n = |x|$. Queremos decidir si existe $u \in \{0, 1\}^*$ tal que $x = uu$. Construimos una MT determinista con cinta de entrada de solo lectura y una cinta de trabajo.

Algoritmo 5 MT para $L = \{uu : u \in \{0, 1\}^*\}$

Entrada: Una palabra $x \in \{0, 1\}^*$.

Calcular $n = |x|$ en binario usando un contador.

Si n es impar **entonces**

Rechazar.

Fin Si

Calcular $m = n/2$ eliminando el último bit 0 de la representación binaria de n .

Para $i = 1, \dots, m$ **hacer**

Mover el cabezal de entrada hasta la posición i y guardar el símbolo leído en una variable a .

Mover el cabezal de entrada hasta la posición $i + m$ y guardar el símbolo leído en una variable b .

Si $a \neq b$ **entonces**

Rechazar.

Fin Si

Fin Para

Aceptar.

La máquina acepta exactamente las palabras de la forma uu , ya que compara, posición a posición, la primera mitad de x con la segunda mitad. Si todas las posiciones coinciden, entonces $x = uu$, con u igual a la primera mitad de x ; si alguna no coincide, entonces x no puede ser de esa forma.

Veamos el espacio usado. La máquina solo necesita almacenar los contadores n , m e i , además de dos símbolos $a, b \in \{0, 1\}$. Como estos contadores toman valores menores o iguales que n , cada uno ocupa $O(\log n)$ casillas. Además, calcular $m = n/2$ no requiere realizar una división general: como ya se ha comprobado que n es par, su representación binaria acaba en 0, y dividir por 2 consiste simplemente en eliminar ese último bit. Por tanto, ese cálculo no aumenta el orden de espacio usado. Así, el espacio total es $O(\log n)$.

Luego

$$L = \{uu : u \in \{0, 1\}^*\}$$

se reconoce en espacio logarítmico.

Ejercicio 3 (2.4 puntos). Razona si son verdaderas o falsas las siguientes afirmaciones:

1. Una MT con una cinta ilimitada en ambas direcciones se puede simular siempre con una MT con una cinta ilimitada solo por la derecha y usando el mismo número de pasos.

Falsa. Una MT con cinta ilimitada en ambas direcciones sí puede simularse con una cinta limitada solo por la derecha usando dos pistas, colocando en una pista las posiciones $0, 1, 2, \dots$ y en otra las posiciones $-1, -2, \dots$. Sin embargo, no se garantiza que use exactamente el mismo número de pasos: la construcción introduce estados auxiliares para preparar la cinta y gestionar qué pista está activa. Por tanto, la simulación existe, pero no necesariamente conserva exactamente el número de pasos.

2. Para que una MT tenga una complejidad en espacio de $O(n^2)$ tiene que tener más de una cinta para no escribir en la cinta de entrada.

Falsa. Como contraejemplo, consideramos una MT de una sola cinta que, con entrada $x \in \{0, 1\}^*$ de longitud n , escribe n copias de x a la derecha de la entrada.

De forma algorítmica:

Algoritmo 6 MT de una cinta que usa espacio $O(n^2)$

Entrada: Una palabra $x \in \{0, 1\}^*$.

Recorre la entrada de izquierda a derecha.

Para cada símbolo de la entrada **hacer**

Vuelve al principio de la entrada.

Copia toda la palabra x al final de la zona ya escrita.

Fin Para

Acepta.

Como el bucle exterior se ejecuta una vez por cada símbolo de x , se ejecuta n veces. En cada iteración copia una palabra de longitud n . Por tanto, al final ha escrito n copias de x , es decir, ha usado al menos n^2 casillas adicionales. Esta MT tiene una sola cinta, luego no es necesario tener más de una cinta para tener complejidad espacial $O(n^2)$.

3. Nuestro conocimiento actual es compatible con el hecho de que $\mathbf{P} \neq \mathbf{NP}$ y $\mathbf{NP} = \mathbf{CoNP}$.

Verdadera. Es compatible con el conocimiento actual que $\mathbf{P} \neq \mathbf{NP}$ y, a la vez, $\mathbf{NP} = \mathbf{CoNP}$. Sabemos que $\mathbf{P} \subseteq \mathbf{NP}$ y $\mathbf{P} \subseteq \mathbf{CoNP}$, pero no está demostrado ni que $\mathbf{P} = \mathbf{NP}$, ni que $\mathbf{NP} = \mathbf{CoNP}$, ni que $\mathbf{NP} \neq \mathbf{CoNP}$.

4. Está demostrado que la tesis de Church-Turing es cierta.

Falsa. “Es una tesis que no puede ser comprobada matemáticamente si no se da una definición formal de efectivamente calculable, pero eso es precisamente lo que intenta hacer la tesis de Church Turing. Es una tesis que es aceptada como cierta. Aunque consideremos modelos de computación cuántica, no permitirían hacer cálculos más allá de los realizados por una MT (aunque quizá en menos tiempo que una MT).”

Ejercicio 4 (1.3 puntos). Enuncia los problemas de la suma. ¿Qué sabes sobre su complejidad? Razona de forma breve la respuesta.

El problema de la SUMA se enuncia como sigue: dados un conjunto finito A , una función de tamaños $s : A \rightarrow \mathbb{N}$, y un entero B , determinar si existe un subconjunto $A' \subseteq A$ tal que

$$\sum_{a \in A'} s(a) = B.$$

Este problema está en **NP**, porque basta elegir no deterministamente el subconjunto A' y comprobar en tiempo polinómico si la suma de sus tamaños es exactamente B .

Además, SUMA es **NP**-completo. Según lo que se ha visto en teoría se ha demostrado reduciendo ACTRI a SUMA: para cada tripleta se construye un número, y se fija un valor B de forma que elegir números que suman B equivale exactamente a elegir un acoplamiento de tripletas. Como ACTRI es **NP**-completo y $\text{ACTRI} \propto \text{SUMA}$, se concluye que SUMA es **NP**-completo.

También aparece relacionado el problema de la **PARTICIÓN**: dado un conjunto C con tamaños $s : C \rightarrow \mathbb{N}$, determinar si existe $C' \subseteq C$ tal que

$$\sum_{a \in C'} s(a) = \sum_{a \in C \setminus C'} s(a).$$

En teoría hemos visto cómo reducir SUMA a PARTICIÓN, por lo que PARTICIÓN también es **NP**-completo.

Ejercicio 5 (1.3 puntos). Define la clase **EXP**, ¿qué relación existe con las clases **PESPACIO** y **P**?

La clase **EXP** es la clase de los lenguajes decidibles en tiempo exponencial determinista:

$$\mathbf{EXP} = \bigcup_{j>0} \mathbf{TIEMPO}(2^{n^j}).$$

Según se ha visto en teoría:

$$\mathbf{P} = \bigcup_{j>0} \mathbf{TIEMPO}(n^j), \quad \mathbf{PESPACIO} = \bigcup_{j>0} \mathbf{ESPACIO}(n^j).$$

La relación entre ellas es

$$\mathbf{P} \subseteq \mathbf{PESPACIO} \subseteq \mathbf{EXP}.$$

En efecto, si un algoritmo trabaja en tiempo polinómico, entonces no puede usar más que espacio polinómico, luego

$$\mathbf{P} \subseteq \mathbf{PESPACIO}.$$

Por otro lado, si una MT usa espacio polinómico, entonces solo tiene exponencialmente muchas configuraciones posibles; por tanto, se puede decidir en tiempo exponencial si acepta o no, luego

$$\mathbf{PESPACIO} \subseteq \mathbf{EXP}.$$

Además, por el teorema de jerarquía en tiempo se sabe que

$$\mathbf{P} \neq \mathbf{EXP}.$$

Por tanto, al menos una de las dos inclusiones anteriores es estricta:

$$\mathbf{P} \subsetneq \mathbf{PESPACIO} \quad \text{o} \quad \mathbf{PESPACIO} \subsetneq \mathbf{EXP},$$

pero no se sabe cuál.

Ejercicio 6 (1.3 puntos). Elige entre una de las siguientes opciones:

- Describe una reducción Turing de $\text{FSAT}(C)$ a $\text{SAT}(C)$.

El problema $\text{SAT}(C)$ decide si un conjunto de cláusulas C es satisfacible. El problema $\text{FSAT}(C)$ consiste en encontrar una asignación satisfactoria para C , si existe.

Suponemos que tenemos un oráculo para SAT . Construimos una reducción Turing para FSAT :

Algoritmo 7 Reducción Turing de FSAT a SAT

Entrada: Un conjunto de cláusulas C con variables $x_1, \dots, x_n$.

Si $\text{SAT}(C)$ responde NO **entonces**

 Responder que no existe asignación satisfactoria.

Fin Si

Para $i = 1, \dots, n$ **hacer**

 Sea $C_0 = C \cup \{\neg x_i\}$.

Si $\text{SAT}(C_0)$ responde SI **entonces**

 Fijar $x_i = 0$ y sustituirlo en C .

Si no

 Fijar $x_i = 1$ y sustituirlo en C .

Fin Si

Fin Para

Devolver la asignación obtenida.

El algoritmo hace a lo sumo $n + 1$ llamadas al oráculo SAT . En cada paso mantiene una fórmula satisfacible: si al fijar $x_i = 0$ sigue siendo satisfacible, tomamos $x_i = 0$; si no, como la fórmula anterior era satisfacible, necesariamente existe una solución con $x_i = 1$. Por tanto, al final se obtiene una asignación satisfactoria de C . Luego

$$\text{FSAT} \propto_T \text{SAT}.$$

- Sean los problemas de la existencia de un circuito hamiltoniano, $\text{HAMILTONIANO}(G)$, y el problema de buscar un circuito hamiltoniano cuando este exista, $\text{FHAMILTONIANO}(G)$. Describe una reducción Turing de $\text{FHAMILTONIANO}(G)$ a $\text{HAMILTONIANO}(G)$.

Suponemos que tenemos un oráculo para **HAMILTONIANO**. Construimos una reducción Turing para **FHAMILTONIANO**:

Algoritmo 8 Reducción Turing de **FHAMILTONIANO** a **HAMILTONIANO**

Entrada: Un grafo no dirigido $G = (V, E)$.
Si $\text{HAMILTONIANO}(G)$ responde NO **entonces**
 Responder que no existe circuito hamiltoniano.
Fin Si
 $G' \leftarrow G$.
Para cada arista $e \in E$ **hacer**
 Sea $G_e = G' - e$.
 Si $\text{HAMILTONIANO}(G_e)$ responde SI **entonces**
 $G' \leftarrow G_e$.
 Fin Si
Fin Para
Devolver el circuito formado por las aristas de G' .

El algoritmo elimina una arista siempre que, al quitarla, el grafo sigue teniendo algún circuito hamiltoniano. Por tanto, mantiene invariante que G' tiene un circuito hamiltoniano. Al final, ninguna arista sobrante puede eliminarse sin destruir todos los circuitos hamiltonianos; en particular, las aristas que quedan contienen exactamente un circuito hamiltoniano. Ese circuito se devuelve. Luego

$$\text{FHAMILTONIANO} \propto_T \text{HAMILTONIANO}.$$